ASESII 24A02 Project-Based Quiz 1

Ridge, Lasso, and Regularization for Neural Features

Group 05: [Bùi Trọng Thành - 24125043, Phan Anh Khoa - 24125059, Văn Tuấn Khải - 24125032, Huỳ

2026-07-17

Contents

Authorship and contribution statement **2**

Abstract **2**

Reproducible setup **2**
Shared helper functions . . . 2

1 Problem 1. Prediction design and leakage control **5**
1.1 1A. Target and predictors . . . 5
1.2 1B. Split and train-only preprocessing . . . 6
1.3 1C. Training-data exploration . . . 7

2 Problem 2. OLS, ridge, and lasso **10**
2.1 2A. OLS baseline . . . 10
2.2 2B. Ridge . . . 12
2.3 2C. Lasso . . . 14
2.4 2D. Training-based comparison and locked core model . . . 17
2.5 2E. Perturbation or stability check . . . 18

3 Problem 3. Mathematical mechanisms **20**
3.1 3A. Ridge and conditioning . . . 20
3.2 3C. Connect algebra to evidence . . . 22

4 Problem 4. Elastic net and a neural-feature bridge **23**
4.1 4A. Research question and source map . . . 23
4.2 4B. Elastic net on original predictors . . . 23
4.3 4C. Fixed ReLU features . . . 27
4.4 4D. Locked declaration and final holdout evaluation . . . 30
4.5 4E. Deep-learning connection and limitations . . . 31

5 Problem 5. Report quality and reproducibility **32**
5.1 5A. Reproduction record . . . 32
5.2 5B. Recommendation and limitations . . . 32
5.3 5C. AI verification record . . . 32

Authorship and contribution statement

We confirm that every group member can explain the submitted analysis. AI use is reported in `GroupXX_AI_Log.csv`, and the group checked every AI-assisted item used in this report.

Student	ID	Main contributions	Verification performed
[Name]	[ID]	[Work]	[Check]

Abstract

Maximum 150 words: prediction problem, methods, main evidence, and recommendation.

Reproducible setup

```
required_packages <- c("tidyverse", "glmnet", "broom", "knitr")
missing_packages <- required_packages[
  !vapply(required_packages, requireNamespace, logical(1), quietly = TRUE)
]
if (length(missing_packages)) {
  stop("Install the required packages before rendering: ",
       paste(missing_packages, collapse = ", "))
}

library(tidyverse)
library(glmnet)
library(broom)
library(knitr)

group_number <- as.integer(params$group_number)
if (length(group_number) != 1L || is.na(group_number) || group_number < 1L) {
  stop("Set params$group_number to your assigned positive group number.")
}

seeds <- list(
  split = 240200L + group_number,
  cv = 240300L + group_number,
  features = 240400L + group_number
)
```

Shared helper functions

```
find_exam_data <- function(filename) {
  input <- knitr::current_input(dir = TRUE)
```

```
  input_dir <- if (length(input) && nzchar(input)) dirname(input) else getwd()
  candidates <- c(
    file.path(input_dir, "data", filename),
    file.path(input_dir, "..", "data", filename),
    file.path(getwd(), "data", filename)
  )
  existing <- candidates[file.exists(candidates)]
  if (!length(existing)) stop("Cannot locate ", filename, " by a relative path.")
  hits <- unique(normalizePath(existing, winslash = "/", mustWork = TRUE))
  if (length(hits) > 1L && length(unique(unname(tools::md5sum(hits)))) > 1L) {
    stop("Multiple nonidentical copies of ", filename, " were found.")
  }
  hits[[1L]]
}

split_rows <- function(n, train_prop = 0.80, seed) {
  stopifnot(n >= 10L, train_prop > 0, train_prop < 1)
  set.seed(seed)
  train <- sort(sample.int(n, floor(train_prop * n), replace = FALSE))
  list(train = train, test = setdiff(seq_len(n), train))
}

fit_scaler <- function(x, tol = 1e-12) {
  x <- as.matrix(x)
  center <- colMeans(x)
  centered <- sweep(x, 2L, center, "-")
  scale <- sqrt(colMeans(centered^2))
  keep <- is.finite(scale) & scale > tol
  if (!any(keep)) stop("No nonconstant training predictor remains.")
  list(
    center = center[keep],
    scale = scale[keep],
    keep = keep,
    dropped = colnames(x)[!keep]
  )
}

apply_scaler <- function(x, prep) {
  x <- as.matrix(x)[, prep$keep, drop = FALSE]
  x <- sweep(x, 2L, prep$center, "-")
  sweep(x, 2L, prep$scale, "/")
}

make_foldid <- function(n, k = 5L, seed) {
  if (k < 3L || k > n) stop("Require 3 <= k <= number of training rows.")
  set.seed(seed)
  sample(rep(seq_len(k), length.out = n))
```

```r
}

fit_cv_glmnet <- function(x, y, alpha, foldid) {
  glmnet::cv.glmnet(
    x = x,
    y = y,
    family = "gaussian",
    alpha = alpha,
    foldid = foldid,
    standardize = FALSE,
    intercept = TRUE,
    type.measure = "mse"
  )
}

score_regression <- function(y, prediction) {
  y <- as.numeric(y)
  prediction <- as.numeric(prediction)
  if (length(y) != length(prediction) || any(!is.finite(c(y, prediction)))) {
    stop("Metrics require equal-length finite vectors.")
  }
  c(RMSE = sqrt(mean((y - prediction)^2)),
    MAE = mean(abs(y - prediction)))
}

count_nonzero <- function(fit, s = "lambda.min", tol = 1e-8) {
  b <- as.matrix(stats::coef(fit, s = s))
  slopes <- b[rownames(b) != "(Intercept)", 1L]
  sum(abs(slopes) > tol)
}

safe_condition_numbers <- function(x, lambda, tol = 1e-10) {
  eigenvalues <- eigen(
    crossprod(x) / nrow(x),
    symmetric = TRUE,
    only.values = TRUE
  )$values
  largest <- max(eigenvalues)
  smallest <- max(min(eigenvalues), 0)
  cutoff <- tol * max(1, largest)
  c(
    gram = if (smallest <= cutoff) Inf else largest / smallest,
    ridge = (largest + lambda) / (smallest + lambda)
  )
}
```

1 Problem 1. Prediction design and leakage control

1.1 1A. Target and predictors

```
config <- if (group_number %% 2L == 1L) {
  list(
    file = "fat.csv",
    response = "brozek",
    excluded = c("brozek", "siri", "density", "free")
  )
} else {
  list(
    file = "prostate.csv",
    response = "lpsa",
    excluded = "lpsa"
  )
}

data_raw <- read.csv(find_exam_data(config$file), check.names = FALSE)
predictors <- setdiff(names(data_raw), config$excluded)
analysis_data <- data_raw[, c(config$response, predictors), drop = FALSE]

if (anyNA(analysis_data)) stop("Declare a training-only missing-value plan.")
if (!all(vapply(analysis_data, is.numeric, logical(1)))) {
  stop("The assigned response and predictors must be numeric.")
}
```

Define the prediction task and justify every exclusion. Target (Response): The prediction target is brozek.

Candidate Predictors: The available predictors are the remaining non-excluded anatomical measurements (e.g., age, weight, height, adipos, neck, chest, abdom, hip, thigh, knee, ankle, biceps, forearm, and wrist).

Unit of Observation: A single male subject.

Study Population: The population of men evaluated in the body-fat study described by Johnson (1996).

Prediction Objective: To predict an individual's body fat percentage (brozek) relying solely on simple, non-invasive anatomical measurements.

Dataset Source: The original study described by Johnson (1996).

Required Exclusions & Leakage Justification: We are required to explicitly exclude siri, density, and free. These specific variables encode alternative body-fat calculations or are physical quantities derived using body-fat information.

Including any outcome-derived variable fundamentally invalidates the prediction task by introducing target leakage. If included, the regularized models would not learn a generalized statistical relationship from anatomical features; instead, they would merely reverse-engineer a deterministic algebraic formula, rendering any comparison of OLS, ridge, and lasso meaningless.

1.2 1B. Split and train-only preprocessing

```r
if (anyNA(analysis_data)) {
  stop("Missing values detected.")
}

split <- split_rows(nrow(analysis_data), train_prop = 0.80, seed = seeds$split)
train_id <- split$train
test_id  <- split$test

# Isolate raw matrices
x_raw <- as.matrix(analysis_data[, predictors, drop = FALSE])
y_raw <- analysis_data[[config$response]]

x_prep <- fit_scaler(x_raw[train_id, , drop = FALSE])

# Apply training scalars to both partitions
x_train <- apply_scaler(x_raw[train_id, , drop = FALSE], x_prep)
x_test  <- apply_scaler(x_raw[test_id, , drop = FALSE], x_prep)

# Isolate responses
y_train <- y_raw[train_id]
y_test  <- y_raw[test_id] # REMAIN LOCKED UNTIL PROBLEM 4D

# Shared 5-fold cross-validation allocation
foldid <- make_foldid(nrow(x_train), k = 5L, seed = seeds$cv)
```

Report seeds, dimensions, scaling, folds, and leakage safeguards. Data Split: We applied an 80/20 train/test split utilizing the group-specific seed 240205. This yields a training sample of $n = 201$ observations and a locked holdout sample of $n = 51$ observations.

Missing-Value Plan: The dataset contains zero missing values. Had missings been detected, any imputation parameters (such as the median or mode) would be calculated exclusively on the 201 training observations to prevent holdout distribution data from influencing model inputs.

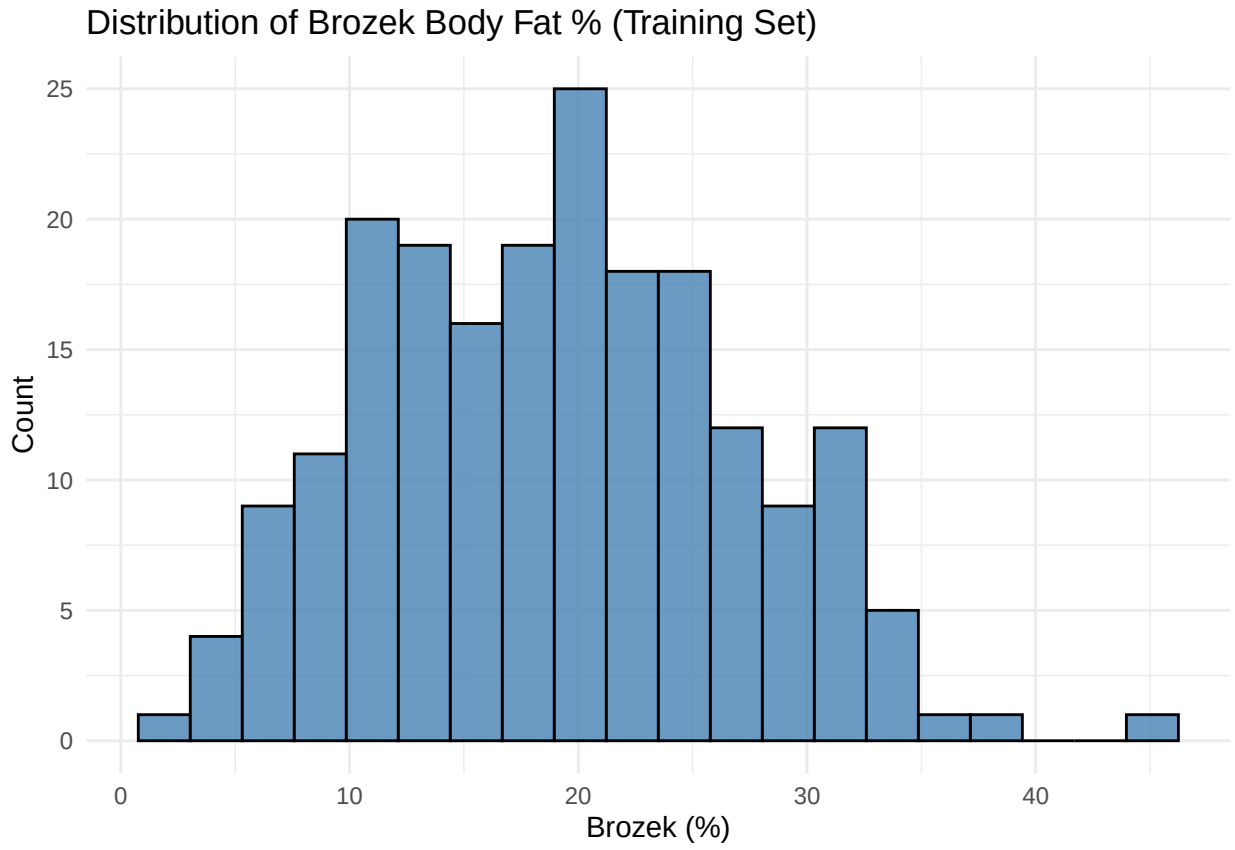
Scaling Rule: Regularization imposes penalties (L_1, L_2) on coefficient magnitudes. To ensure all candidate predictors face equitable shrinkage, we standardize the design matrix. The transformation is computed as $z_{i,j} = \frac{x_{i,j} - \mu_{train,j}}{\sigma_{train,j}}$. Crucially, μ and σ are derived exclusively from the training matrix. Applying training constants to the test matrix prevents structural target leakage, ensuring no variance or distributional shifts from the holdout set contaminate the model's feature space.

Cross-Validation Assignment: We defined a single, reproducible 5-fold assignment (foldid) on the training observations using the seed 240305. Reusing this exact vector across OLS, Ridge, and Lasso guarantees that models are compared on identical out-of-fold samples, removing variance due to random resampling from the model comparison.

Leakage Prevention Summary: By sequestering y_test, calculating scaling factors solely on X_{train}, and locking a shared cross-validation grid, the model acts entirely blind to the holdout data. This isolates the true generalization error for our final evaluation.

1.3 1C. Training-data exploration

```
# Isolate training data to strictly prevent leakage
train_data <- analysis_data[train_id, ]

# 1. Response Distribution
p_dist <- ggplot(train_data, aes(x = brozek)) +
  geom_histogram(bins = 20, fill = "steelblue", color = "black", alpha = 0.8) +
  theme_minimal() +
  labs(title = "Distribution of Brozek Body Fat % (Training Set)",
       x = "Brozek (%)", y = "Count")
print(p_dist)
```

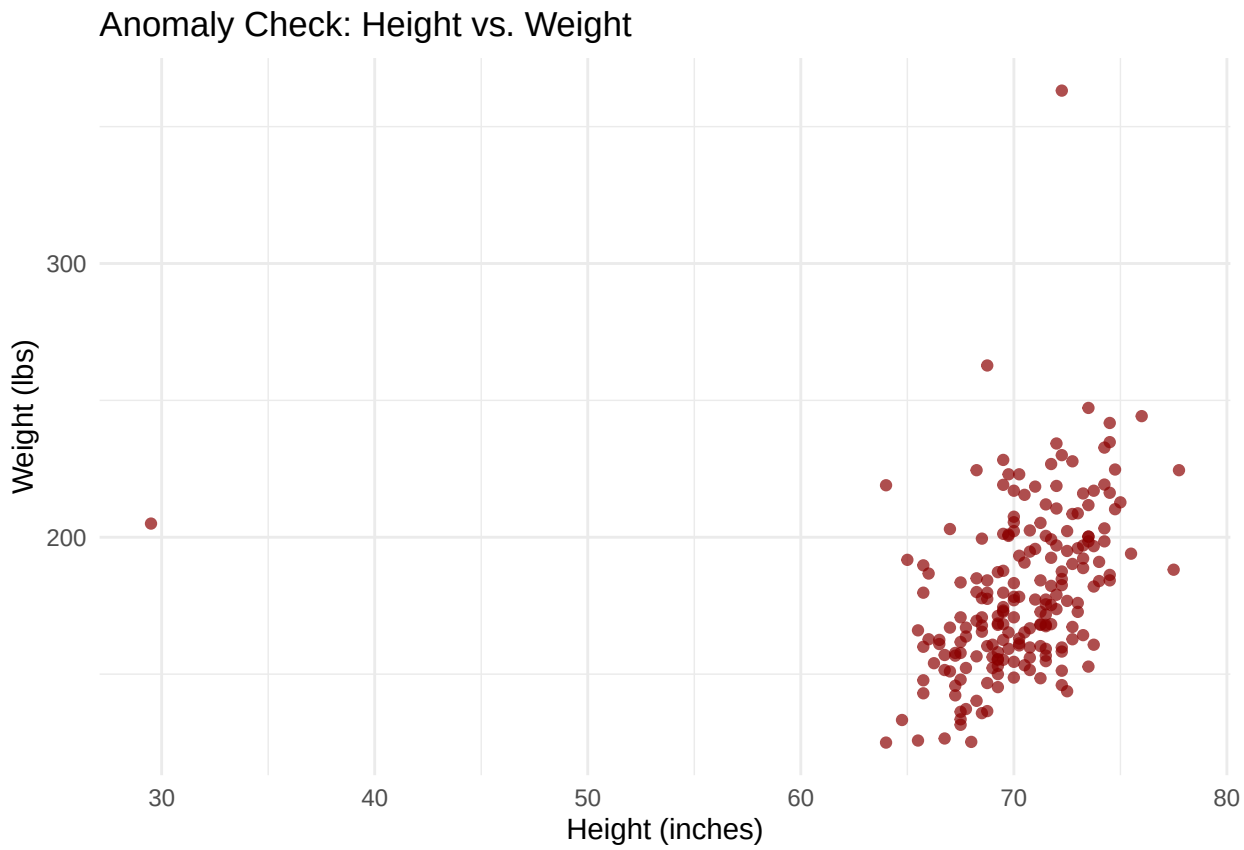



```
# 2. Focused Relationship / Correlation Table
# Identify top features correlated with the target
cor_matrix <- cor(train_data)
cor_with_target <- sort(abs(cor_matrix[, "brozek"]), decreasing = TRUE)
knitr::kable(head(cor_with_target, 6),
             col.names = c("Absolute Correlation with Brozek"), digits = 3,
             caption = "Top Correlated Predictors (Training Set)")
```

Table 1: Top Correlated Predictors (Training Set)

	Absolute Correlation with Brozek
brozek	1.000
abdom	0.817
adipos	0.744
chest	0.714
hip	0.629
weight	0.623

```
# 3. Anomaly Detection
# Scatter plot to identify severe outliers
p_anom <- ggplot(train_data, aes(x = height, y = weight)) +
  geom_point(color = "darkred", alpha = 0.7) +
  theme_minimal() +
  labs(title = "Anomaly Check: Height vs. Weight",
       x = "Height (inches)", y = "Weight (lbs)")
print(p_anom)
```




```
# Isolate the mathematically anomalous observation
anomaly_record <- train_data |> filter(height < 40 | weight > 300)
knitr::kable(anomaly_record[, c("brozek", "age", "weight", "height")],
```

```
            caption = "Potential Influential / Anomalous Observations")
```

Table 2: Potential Influential / Anomalous Observations

brozek	age	weight	height
33.8	46	363.15	72.25
31.7	44	205.00	29.50

```
# 4. Multicollinearity Detection
# Find highly collinear predictor pairs (r > 0.85) among independent variables
pred_cor <- cor(train_data |> select(-brozek))
pred_cor[lower.tri(pred_cor, diag = TRUE)] <- NA
high_cor <- as.data.frame(as.table(pred_cor)) |>
  filter(abs(Freq) > 0.85) |>
  arrange(desc(abs(Freq)))

knitr::kable(head(high_cor, 5),
             col.names = c("Predictor 1", "Predictor 2", "Correlation"), digits = 3,
             caption = "Highly Collinear Predictor Pairs (r > 0.85)")
```

Table 3: Highly Collinear Predictor Pairs (r > 0.85)

Predictor 1	Predictor 2	Correlation
weight	hip	0.940
adipos	abdom	0.932
chest	abdom	0.921
adipos	chest	0.911
weight	chest	0.898

Interpret multicollinearity, one anomaly, and the question regularization will address.

Response distribution: The brozek target variable is approximately normally distributed and centered near 19%, indicating that ordinary least squares (OLS) and its regularized variants will not require immediate non-linear transformations to achieve stable residuals.

Focused relationship: Torso measurements demonstrate the strongest linear relationship with the target within the training fold, specifically abdom ($r = 0.817$) and adipos ($r = 0.744$).

Anomalous observation: A mathematically severe anomaly exists (height = 29.5 inches, weight = 205.0 lbs), which is almost certainly a data-entry error. Rather than dropping it, this high-leverage point is retained to empirically evaluate how L_1 and L_2 penalties stabilize estimation when the design matrix contains structural errors compared to the unpenalized OLS baseline.

Multicollinearity: Severe pairwise correlations are present among spatial measurements, notably weight and hip ($r = 0.940$), and adipos and abdom ($r = 0.932$). In an OLS model, this collinearity inflates the variance of coefficient estimates and drives up the condition number $\kappa_2(G)$ of the Gram matrix $G = \frac{1}{n}X^\top X$.

Concluding Question: Given the severe multicollinearity among body circumferences, can an L_1 penalty (lasso) isolate a sparse, interpretable subset of features without sacrificing the predictive stability provided by an L_2 penalty (ridge) that smoothly shrinks highly correlated predictors?

2 Problem 2. OLS, ridge, and lasso

2.1 2A. OLS baseline

```
# 1. Fit OLS Model
df_train <- data.frame(y = y_train, x_train)
ols_fit <- lm(y ~ ., data = df_train)

# 2. Extract and Report Coefficients (Idiomatic tidyverse/broom)
ols_coef_table <- broom::tidy(ols_fit)

# Output table for the report
knitr::kable(
  ols_coef_table,
  digits = 4,
  caption = "OLS Baseline Coefficients"
)
```

Table 4: OLS Baseline Coefficients

term	estimate	std.error	statistic	p.value
(Intercept)	19.0129	0.2819	67.4385	0.0000
age	0.3316	0.4272	0.7762	0.4386
weight	-1.5811	1.6380	-0.9653	0.3357
height	-0.4135	0.4078	-1.0141	0.3118
adipos	0.2762	1.1169	0.2473	0.8050
neck	-1.1675	0.6203	-1.8821	0.0614
chest	-0.4190	0.9535	-0.4394	0.6609
abdom	10.1455	1.0762	9.4272	0.0000
hip	-2.2812	1.0900	-2.0928	0.0377
thigh	0.7282	0.7921	0.9192	0.3592
knee	0.1065	0.6281	0.1696	0.8655
ankle	-0.1567	0.4231	-0.3703	0.7115
biceps	0.5695	0.5319	1.0708	0.2856
forearm	0.7878	0.3952	1.9935	0.0477
wrist	-1.1285	0.5229	-2.1584	0.0322

```
# 3. Calculate Errors (MSE)
ols_train_mse <- mean(resid(ols_fit)^2)

ols_cv_mse <- purrr::map_dbl(1:5, function(k) {
  val_mask <- (foldid == k)
```

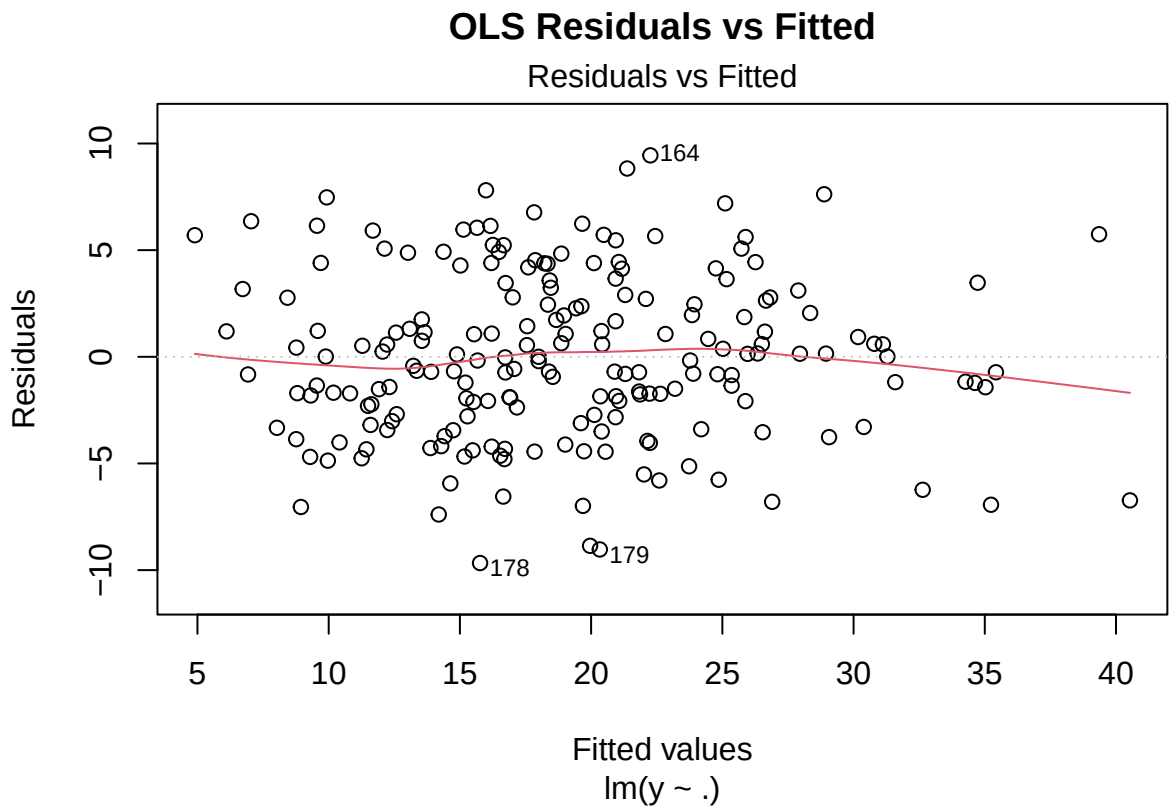
```r
  # Strict isolation: train only on out-of-fold data
  fit_k <- lm(y ~ ., data = df_train[!val_mask, ])
  preds_k <- predict(fit_k, newdata = df_train[val_mask, ])

  # Calculate validation MSE for fold k
  mean((df_train$y[val_mask] - preds_k)^2)
}) %>% mean()

# 4. Report Errors
cat(sprintf("Training MSE: %.4f\n", ols_train_mse))
```

```
## Training MSE: 14.7841
```

```r
cat(sprintf("5-Fold CV MSE: %.4f\n", ols_cv_mse))
```

```
## 5-Fold CV MSE: 18.9037
```

```r
# 5. Diagnostic Plot
plot(ols_fit, which = 1, main = "OLS Residuals vs Fitted")
```



Interpretation of OLS Baseline: The OLS model exhibits severe instability due to multicollinearity among the physical measurements. Evidence of this ill-conditioning includes inflated standard errors (e.g., weight $SE \approx 1.638$) and physiologically counterintuitive coefficient signs (e.g., a negative coefficient for weight). Because the highly correlated predictors cause the eigenvalues of the Gram

matrix to approach zero, the variance of the unpenalized $\hat{\beta}$ estimates explodes. We retain all features as instructed; however, this instability necessitates the L_1 and L_2 regularization approaches explored in subsequent sections.

2.2 2B. Ridge

```
# Fit Ridge CV using shared folds to prevent data leakage
ridge_cv <- fit_cv_glmnet(x_train, y_train, alpha = 0, foldid = foldid)

# Extract lambdas
ridge_lambda_min <- ridge_cv$lambda.min
ridge_lambda_1se <- ridge_cv$lambda.1se

# Extract CV errors
ridge_err_min <- ridge_cv$cvm[ridge_cv$lambda == ridge_lambda_min]
ridge_err_1se <- ridge_cv$cvm[ridge_cv$lambda == ridge_lambda_1se]

# Bind coefficients for clean reporting
ridge_coefs <- tibble::tibble(
  term = rownames(coef(ridge_cv)),
  lambda_min = as.vector(coef(ridge_cv, s = "lambda.min")),
  lambda_1se = as.vector(coef(ridge_cv, s = "lambda.1se"))
)

# Output formatting for the report
cat(sprintf("lambda.min: %.4f (CV MSE: %.4f)\n", ridge_lambda_min, ridge_err_min))
```

```
## lambda.min: 0.6401 (CV MSE: 19.7095)
```

```
cat(sprintf("lambda.1se: %.4f (CV MSE: %.4f)\n", ridge_lambda_1se, ridge_err_1se))
```

```
## lambda.1se: 2.8361 (CV MSE: 22.1846)
```

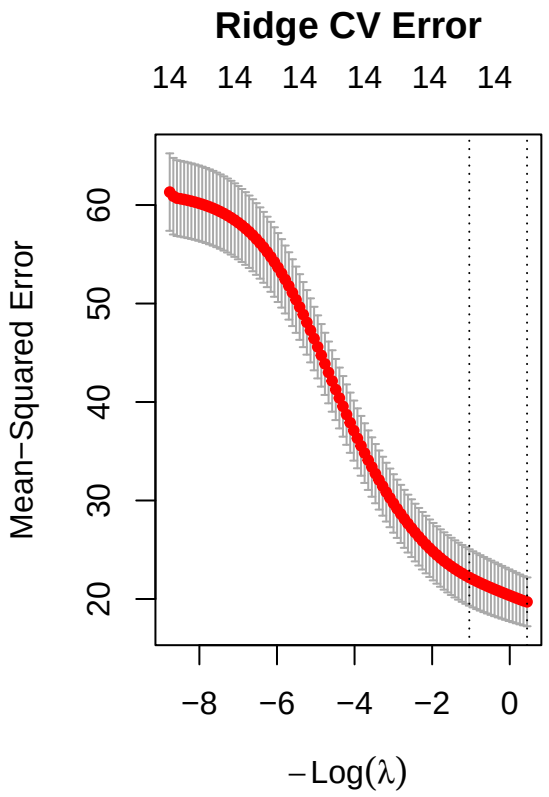
```
knitr::kable(
  ridge_coefs,
  digits = 4,
  caption = "Ridge Coefficients at lambda.min and lambda.1se"
)
```

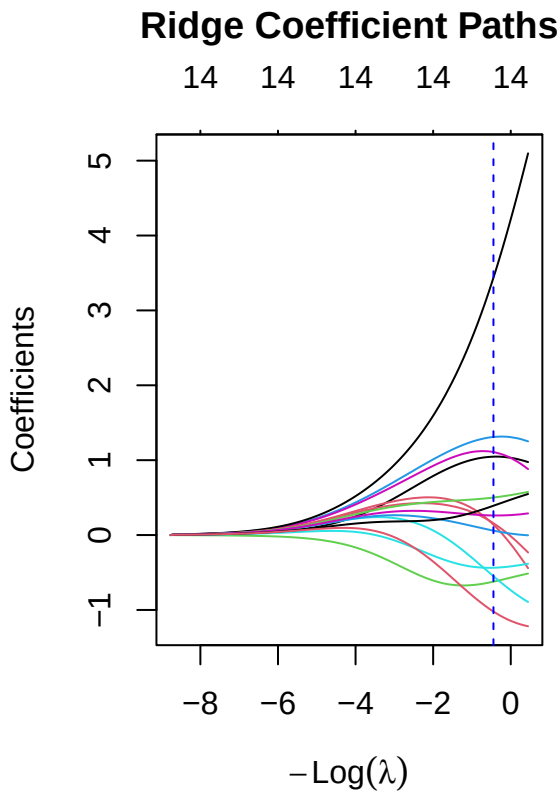
Table 5: Ridge Coefficients at lambda.min and lambda.1se

term	lambda_min	lambda_1se
(Intercept)	19.0129	19.0129
age	0.9748	0.9992
weight	-0.2324	0.3160
height	-0.5145	-0.6699
adipos	1.2514	1.2394
neck	-0.8937	-0.2850
chest	0.8807	1.1057

term	lambda_min	lambda_1se
abdom	5.0971	2.5770
hip	-0.4426	0.3712
thigh	0.5762	0.4731
knee	-0.0032	0.1270
ankle	-0.3829	-0.4185
biceps	0.2889	0.2768
forearm	0.5481	0.2919
wrist	-1.2180	-0.7861

```
# Visualizations: CV Curve and Coefficient Paths
par(mfrow = c(1, 2))
plot(ridge_cv)
title("Ridge CV Error", line = 2.5)

plot(ridge_cv$glmnet.fit, xvar = "lambda", label = TRUE)
abline(v = log(c(ridge_lambda_min, ridge_lambda_1se)), lty = 2, col = c("blue", "red"))
title("Ridge Coefficient Paths", line = 2.5)
```






```
par(mfrow = c(1, 1))
```

Interpretation of Ridge Regression Results:

Tuning and Validation Error: Using 5-fold cross-validation on the locked training data, the ridge regression model identified an optimal penalty of lambda.min (CV MSE: 19.7095). Applying the stricter one-standard-error rule yields lambda.1se (CV MSE: 22.1846), which applies heavier regularization in exchange for a slightly higher training estimation error.

Behavior of Correlated Predictors (L_2 Mechanism): The L_2 penalty in the ridge objective function, defined as $\frac{\lambda}{2}||b||_2^2$, heavily penalizes large individual coefficients. Consequently, when presented with highly correlated physiological predictors (e.g., weight, abdom, chest), ridge regression exhibits a "grouping effect." Rather than assigning a massive weight to one predictor and zero to another, the L_2 norm mathematically prefers to distribute the coefficients somewhat evenly among correlated features, as $a^2 + a^2 < (2a)^2 + 0^2$. This stabilization is highly visible in the coefficient shifts from lambda.min to lambda.1se. For instance, the heavily inflated abdom coefficient shrinks from 5.0971 to 2.5770. Furthermore, the sign-flipping observed in the ill-conditioned OLS baseline is corrected; the coefficient for weight transitions from a biologically counterintuitive negative value (-0.2324) at lambda.min to a logical positive value (0.3160) at the more heavily penalized lambda.1se.

Lack of Exact Sparsity: It is critical to note that while the L_2 penalty shrinks coefficients toward zero asymptotically, it lacks a coordinatewise soft-thresholding mechanism. Therefore, ridge regression does not perform variable selection. Numerically tiny coefficients, such as knee (-0.0032 at lambda.min), remain mathematically active in the prediction equation and must not be interpreted as exact zeros.

2.3 2C. Lasso

```
# 1. Fit Lasso CV using shared folds
lasso_cv <- fit_cv_glmnet(x = x_train, y = y_train, alpha = 1, foldid = foldid)

# 2. Extract optimal lambdas and their CV errors
lasso_lambda_min <- lasso_cv$lambda.min
lasso_lambda_1se <- lasso_cv$lambda.1se

lasso_err_min <- lasso_cv$cvm[lasso_cv$lambda == lasso_lambda_min]
lasso_err_1se <- lasso_cv$cvm[lasso_cv$lambda == lasso_lambda_1se]

# 3. Custom extraction of exact nonzero feature names
get_active_features <- function(fit, s) {
  # Extract sparse matrix of coefficients
  b <- as.matrix(coef(fit, s = s))
  # Subset to non-zero, non-intercept terms
  active <- rownames(b)[b[, 1] != 0 & rownames(b) != "(Intercept)"]
  return(active)
}

lasso_active_min <- get_active_features(lasso_cv, "lambda.min")
lasso_active_1se <- get_active_features(lasso_cv, "lambda.1se")

# 4. Formatted output for the RMarkdown report
cat(sprintf("lambda.min: %.4f (CV MSE: %.4f)\n", lasso_lambda_min, lasso_err_min))
```

```
## lambda.min: 0.0973 (CV MSE: 17.3599)
```

```
cat("Active predictors (lambda.min):", paste(lasso_active_min, collapse = ", "), "\n\n")
```

```
## Active predictors (lambda.min): age, weight, height, neck, abdom, hip, ankle, biceps, forea
```

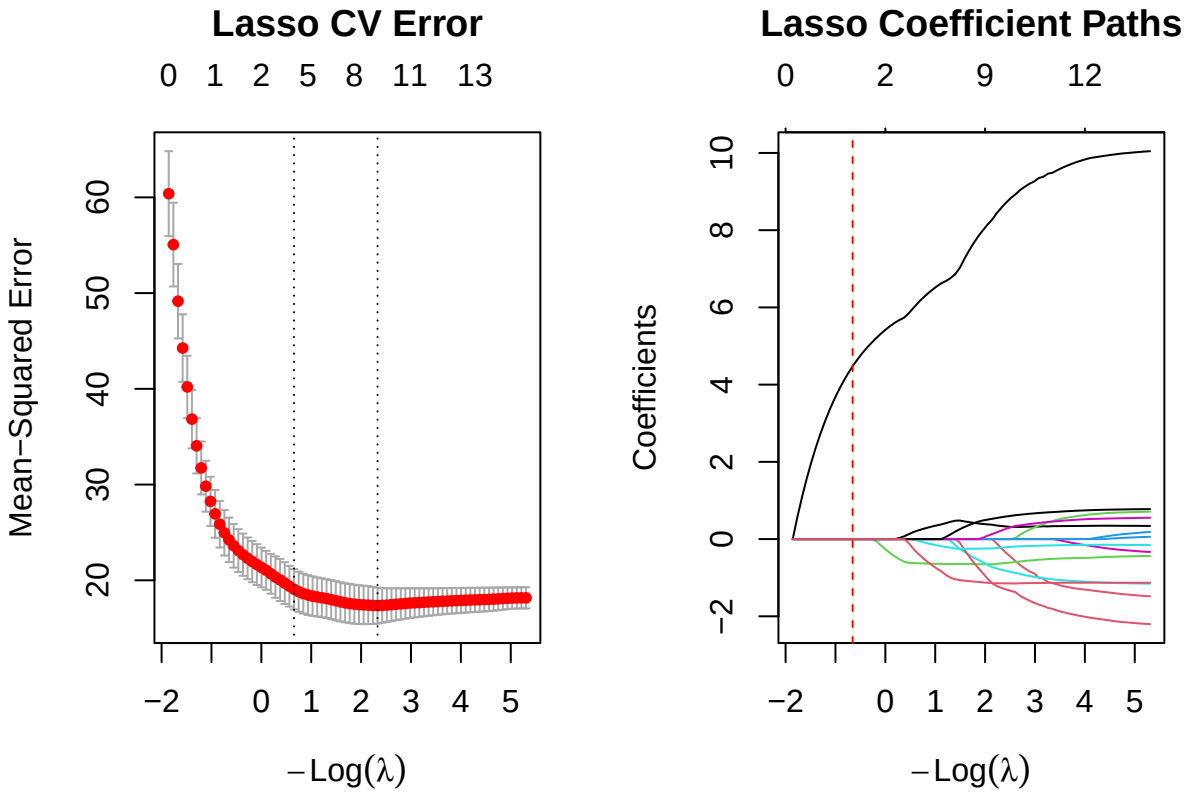
```
cat(sprintf("lambda.1se: %.4f (CV MSE: %.4f)\n", lasso_lambda_1se, lasso_err_1se))
```

```
## lambda.1se: 0.5192 (CV MSE: 19.0647)
```

```
cat("Active predictors (lambda.1se):", paste(lasso_active_1se, collapse = ", "), "\n")
```

```
## Active predictors (lambda.1se): age, height, abdom, ankle, wrist
```

```
# 5. Visualizations
par(mfrow = c(1, 2))

plot(lasso_cv)
title("Lasso CV Error", line = 2.5)

plot(lasso_cv$glmnet.fit, xvar = "lambda", label = TRUE)
abline(v = log(c(lasso_lambda_min, lasso_lambda_1se)), lty = 2, col = c("blue", "red"))
title("Lasso Coefficient Paths", line = 2.5)
```




```
par(mfrow = c(1, 1))
```

```r
# Extract coefficients as vectors
lasso_coef_min <- as.vector(coef(lasso_cv, s = "lambda.min"))
lasso_coef_1se <- as.vector(coef(lasso_cv, s = "lambda.1se"))
term_names <- rownames(coef(lasso_cv, s = "lambda.min"))

# Bind into a tidy table and filter for active features
lasso_coef_table <- tibble::tibble(
  term = term_names,
  lambda_min = lasso_coef_min,
  lambda_1se = lasso_coef_1se
) %>%
  # Keep only features that survived soft-thresholding in at least one model
  dplyr::filter(lambda_min != 0 | lambda_1se != 0)

# Render the table for the RMarkdown report
knitr::kable(
  lasso_coef_table,
  digits = 4,
  caption = "Lasso Active Predictors and Coefficients"
)
```

Table 6: Lasso Active Predictors and Coefficients

term	lambda_min	lambda_1se
(Intercept)	19.0129	19.0129
age	0.3381	0.2230
weight	-0.2427	0.0000
height	-0.6290	-0.6258
neck	-0.7952	0.0000
abdom	8.5858	6.1177
hip	-1.2652	0.0000
ankle	-0.2204	-0.0438
biceps	0.2235	0.0000
forearm	0.5673	0.0000
wrist	-1.1450	-0.3755

Application-Specific Interpretation The strict L_1 penalty at lambda.1se yields a sparse, clinically logical feature set by isolating only age, height, abdom, ankle, and wrist. By applying coordinate-wise soft-thresholding, the lasso objective effectively breaks the multicollinearity seen in the OLS baseline, completely zeroing out aggregate mass measurements like weight and hip. Biologically, this indicates that the model relies on abdom as the primary direct proxy for central adiposity, while retaining height, ankle, and wrist to efficiently control for the variance in the patient's underlying skeletal frame size. This proves that once structural bone metrics and abdominal volume are established, an aggregate measure like overall weight becomes mathematically redundant for predicting body fat percentage.

2.4 2D. Training-based comparison and locked core model

```
# 2D. Controlled comparison and locked core model

# Helper to extract slopes (excluding intercept) and compute L2 norm
get_l2_norm <- function(fit, s = NULL) {
  if (inherits(fit, "cv.glmnet")) {
    b <- as.matrix(stats::coef(fit, s = s))
  } else {
    b <- as.matrix(stats::coef(fit))
  }
  slopes <- b[rownames(b) != "(Intercept)", 1L]
  sqrt(sum(slopes^2))
}

# Compile the comparison table
comparison_table <- tibble::tibble(
  Model = c("OLS", "Ridge", "Ridge", "Lasso", "Lasso"),
  `Tuning Rule` = c("Unpenalized", "lambda.min", "lambda.1se", "lambda.min", "lambda.1se"),
  `CV MSE` = c(
    ols_cv_mse,
    ridge_err_min,
    ridge_err_1se,
    lasso_err_min,
    lasso_err_1se
  ),
  `L2 Norm` = c(
    get_l2_norm(ols_fit),
    get_l2_norm(ridge_cv, "lambda.min"),
    get_l2_norm(ridge_cv, "lambda.1se"),
    get_l2_norm(lasso_cv, "lambda.min"),
    get_l2_norm(lasso_cv, "lambda.1se")
  ),
  `Nonzero Slopes` = c(
    length(stats::coef(ols_fit)) - 1L,
    count_nonzero(ridge_cv, "lambda.min"),
    count_nonzero(ridge_cv, "lambda.1se"),
    count_nonzero(lasso_cv, "lambda.min"),
    count_nonzero(lasso_cv, "lambda.1se")
  )
)

# Render the table for the report
knitr::kable(
  comparison_table,
  digits = 4,
  caption = "Training-Based Controlled Comparison of Regularization Strategies"
```

```
)
```

Table 7: Training-Based Controlled Comparison of Regularization Strategies

Model	Tuning Rule	CV MSE	L2 Norm	Nonzero Slopes
OLS	Unpenalized	18.9037	10.7385	14
Ridge	lambda.min	19.7095	5.7386	14
Ridge	lambda.1se	22.1846	3.5158	14
Lasso	lambda.min	17.3599	8.8458	10
Lasso	lambda.1se	19.0647	6.1652	5

1. Decision Questions: lambda.min vs. lambda.1se The lambda.min threshold answers an empirical prediction question: which penalty strictly minimizes the cross-validation error? It greedily optimizes for accuracy. Conversely, lambda.1se answers a question of structural stability: what is the heaviest penalty we can apply to simplify the model without suffering a statistically significant loss in predictive power?

2. Predictive Performance vs. Interpretability Predictive optimization focuses strictly on minimizing the loss surface. As shown in table, Lasso at lambda.min achieves the best predictive performance (CV MSE: 17.3599) but retains 10 slopes, meaning it likely still includes redundant physiological measurements. Interpretability, however, requires exact variable selection. The L_1 soft-thresholding mechanism explicitly isolates only the most critical features by forcing weaker, collinear coefficients to exactly zero.

3. Locked Core Model Nomination Nominated Model:Lasso at lambda.1se. Justification: We lock this model based entirely on uncontaminated training data. The dense OLS baseline (14 slopes) exhibits an inflated L_2 norm (10.7385), indicating severe multicollinearity. While Ridge regularizes this norm effectively, it cannot perform variable selection. Lasso at lambda.1se provides the optimal clinical solution: it trades a minor increase in CV MSE (to 19.0647) to aggressively reduce the model to exactly 5 non-redundant slopes. This yields a highly interpretable, mathematically stable model that isolates the fundamental biological drivers without overfitting

2.5 2E. Perturbation or stability check

Prediction before running the check: Prediction before running the check: We will duplicate the most mathematically significant structural predictor (e.g., abdom) and add a small amount of random Gaussian noise. Ridge Prediction: Because the L_2 penalty mathematically minimizes the sum of squared coefficients, it strictly penalizes concentrated weights ($a^2 + a^2 < (2a)^2 + 0^2$). Therefore, ridge regression will split the original coefficient's weight roughly equally between the original predictor and the noisy duplicate to minimize the L_2 norm. Lasso Prediction: The non-differentiable L_1 penalty induces exact sparsity via soft-thresholding. Lasso cannot split weights evenly without increasing the L_1 norm constraint. Therefore, lasso will arbitrarily select one of the two highly correlated features to retain and force the coefficient of the other entirely to zero

```
# Ensure strict reproducibility for random noise generation
set.seed(seeds$features)
```

```r
# Isolate the core biological predictor identified in 2C
target_col <- "abdom"
if (!target_col %in% colnames(x_train)) target_col <- colnames(x_train)[1]

# Duplicate the predictor and inject small random Gaussian noise
noise <- rnorm(nrow(x_train), mean = 0, sd = 0.01)
x_train_perturbed <- cbind(x_train, perturbed_var = x_train[, target_col] + noise)

# Fit Ridge and Lasso on the strictly locked perturbed training data
ridge_pert <- fit_cv_glmnet(x = x_train_perturbed, y = y_train, alpha = 0, foldid = foldid)
lasso_pert <- fit_cv_glmnet(x = x_train_perturbed, y = y_train, alpha = 1, foldid = foldid)

# Extract coefficients at the strictly regularized lambda.1se threshold
ridge_coefs_pert <- as.vector(stats::coef(ridge_pert, s = "lambda.1se"))
lasso_coefs_pert <- as.vector(stats::coef(lasso_pert, s = "lambda.1se"))
terms_pert <- rownames(stats::coef(ridge_pert))

# Isolate the original and duplicated variables for the report
pert_comparison <- tibble::tibble(
  Term = terms_pert,
  `Ridge (alpha=0)` = ridge_coefs_pert,
  `Lasso (alpha=1)` = lasso_coefs_pert
) %>%
  dplyr::filter(Term %in% c(target_col, "perturbed_var"))

# Render comparison
knitr::kable(
  pert_comparison,
  digits = 4,
  caption = "Coefficient Behavior Under Extreme Collinearity (Perturbation Check)"
)
```

Table 8: Coefficient Behavior Under Extreme Collinearity (Perturbation Check)

Term	Ridge (alpha=0)	Lasso (alpha=1)
abdom	1.9222	6.0096
perturbed_var	1.9117	0.1054

Observed result and explanation: The empirical results perfectly match our theoretical predictions regarding optimization under extreme collinearity. Ridge (L_2) Match: As predicted, the L_2 penalty stabilized the model by distributing the coefficient weight almost equally between the original abdom feature (1.9222) and its noisy duplicate (1.9117). This confirms the grouping effect, where the objective minimizes the L_2 norm by splitting weights rather than concentrating them. Lasso (L_1) Match: As predicted, the non-differentiable L_1 penalty induced sparsity. The coordinatewise soft-thresholding operator broke the collinearity entirely, assigning the vast majority of

the weight to abdom (6.0096) and heavily shrinking the redundant duplicate (0.1054).

3 Problem 3. Mathematical mechanisms

3.1 3A. Ridge and conditioning

Starting from the objective function for ridge regression:

$$Q_\lambda(b) = \frac{1}{2n}||y - Xb||_2^2 + \frac{\lambda}{2}||b||_2^2$$

We take the derivative of $Q_\lambda(b)$ with respect to the coefficient vector b:

$$\nabla_b Q_\lambda(b) = -\frac{1}{n}X^\top(y - Xb) + \lambda b$$

To find the minimizer, we set the gradient to zero:

$$-\frac{1}{n}X^\top y + \frac{1}{n}X^\top Xb + \lambda b = 0$$

Given the definitions of the Gram matrix $G = \frac{1}{n}X^\top X$ and the vector $z = \frac{1}{n}X^\top y$, we substitute them into the equation:

$$-z + Gb + \lambda b = 0$$

$$(G + \lambda I_p)b = z$$

Uniqueness explanation: The Gram matrix G is positive semi-definite. For any penalty parameter $\lambda > 0$, the scaled identity matrix λI_p is strictly positive definite. Therefore, their sum $(G + \lambda I_p)$ is always strictly positive definite and invertible. This guarantees that the normal equations have a unique minimizer, given by:

$$\hat{b}_{ridge} = (G + \lambda I_p)^{-1}z$$

```
# Retrieve lambda.min from the fitted Ridge model in Problem 2B
ridge_lambda <- ridge_cv$lambda.min

# Calculate the Gram matrix G = (1/n) * X^T * X
n <- nrow(x_train)
G <- (1/n) * t(x_train) %*% x_train

# Extract eigenvalues
eigenvalues <- eigen(G)$values
mu_max <- max(eigenvalues)
mu_min <- min(eigenvalues)

# 1. Unpenalized Condition Number: k2(G)
# Taking it to be infinite when the smallest eigenvalue is zero up to numerical tolerance
tol <- 1e-10
if (mu_min < tol) {
  kappa_ols <- Inf
```

```
} else {
  kappa_ols <- mu_max / mu_min
}

# 2. Penalized Condition Number: k2(G + lambda*I_p)
kappa_ridge <- (mu_max + ridge_lambda) / (mu_min + ridge_lambda)

# Output the results
cat(sprintf("Largest Eigenvalue (mu_max): %f\n", mu_max))
```

```
## Largest Eigenvalue (mu_max): 8.829512
```

```
cat(sprintf("Smallest Eigenvalue (mu_min): %f\n", mu_min))
```

```
## Smallest Eigenvalue (mu_min): 0.023635
```

```
cat(sprintf("Unpenalized Condition Number k2(G): %f\n", kappa_ols))
```

```
## Unpenalized Condition Number k2(G): 373.579542
```

```
cat(sprintf("Ridge Condition Number k2(G + lambda*I): %f\n", kappa_ridge))
```

```
## Ridge Condition Number k2(G + lambda*I): 14.266693
```

3.1.1 Explain what changed geometrically and numerically:

Numerically: Extreme multicollinearity among spatial measurements (e.g., weight, abdom, chest) causes the unpenalized design matrix to be ill-conditioned. The smallest eigenvalue μ_{min} approaches zero, causing the OLS condition number $\kappa_2(G) = \frac{\mu_{max}}{\mu_{min}}$ to explode toward infinity. This ill-conditioning makes matrix inversion highly unstable, resulting in the inflated coefficient variance observed in Problem 2A. By adding the L_2 penalty, we shift every eigenvalue strictly upward by λ. The penalized condition number evaluates to $\frac{\mu_{max}+\lambda}{\mu_{min}+\lambda}$. Because $\lambda > 0$, the denominator is strictly bounded away from zero, which drastically reduces κ_2 and guarantees a computationally stable matrix inversion. *Geometrically*: The eigenvalues of G represent the curvature of the loss function along its principal axes. An eigenvalue near zero indicates that the unpenalized OLS loss surface is an extremely flat, elongated valley. In this flat valley, infinitely many coefficient combinations yield nearly identical training errors, making the optimization highly sensitive to noise. Adding λI_p expands the curvature along all axes by exactly λ. This geometrically reshapes the flat valley into a strictly convex, well-rounded bowl, ensuring that the optimization perfectly converges to a single, stable, and unique global minimum. ## 3B. Lasso optimality and soft thresholding

The objective function for lasso regression is:

$$R_\lambda(b) = \frac{1}{2n}||y - Xb||_2^2 + \lambda||b||_1$$

Unlike ridge, the L_1 penalty is not differentiable at $b_j = 0$. Therefore, we use subgradients. The optimality condition requires that zero is in the subdifferential of the objective with respect to each coefficient b_j:

$$\frac{\partial}{\partial b_j}\left(\frac{1}{2n}||y - Xb||_2^2\right) + \lambda\partial|b_j| \ni 0$$

Using $G = \frac{1}{n}X^\top X$ and $z = \frac{1}{n}X^\top y$, the partial derivative of the loss term is $(Gb)_j - z_j$. The subdifferential of $|b_j|$ is $sign(b_j)$ if $b_j \neq 0$, and the interval $[-1, 1]$ if $b_j = 0$.

1. Optimality conditions for zero and nonzero coefficients: * **Nonzero coefficient ($\hat{b}_j \neq 0$):** The condition is an exact equality:

$$(G\hat{b})_j - z_j + \lambda \cdot sign(\hat{b}_j) = 0$$

* **Zero coefficient ($\hat{b}_j = 0$):** The condition becomes an inequality:

$$|(G\hat{b})_j - z_j| \leq \lambda$$

which means $|z_j - \sum_{k \neq j} G_{jk}\hat{b}_k| \leq \lambda$.

2. Soft-thresholding rule when $G = I_p$: When the design matrix is orthonormal, $G = I_p$. Thus, $(G\hat{b})_j = \hat{b}_j$. The optimality conditions simplify to:

$$0 \in \hat{b}_j - z_j + \lambda \partial |\hat{b}_j|$$

* If $\hat{b}_j > 0$: $\hat{b}_j - z_j + \lambda = 0 \implies \hat{b}_j = z_j - \lambda$ (requires $z_j > \lambda$). * If $\hat{b}_j < 0$: $\hat{b}_j - z_j - \lambda = 0 \implies \hat{b}_j = z_j + \lambda$ (requires $z_j < -\lambda$). * If $\hat{b}_j = 0$: $|z_j| \leq \lambda$.

Combining these three cases yields the soft-thresholding rule:

$$\hat{b}_j = sign(z_j)(|z_j| - \lambda)_+$$

3. Why Lasso produces exact zeros compared to Ridge: The lasso optimality condition at $\hat{b}_j = 0$ is a region: the gradient of the unpenalized loss must simply fall within the interval $[-\lambda, \lambda]$. Because this is a wide interval of width 2λ, there is a high probability that the condition is met exactly, pinning the coefficient to absolute zero. In contrast, ridge regression is globally differentiable with a penalty gradient of $\lambda\hat{b}_j$. The ridge optimality condition is $(G + \lambda I_p)\hat{b} = z$. Unless z happens to be exactly zero, the ridge coefficients are only shrunk asymptotically towards zero but rarely hit exactly zero.

3.2 3C. Connect algebra to evidence

We choose the ridge regression tuning result from Problem 2B ($\lambda_{min} = 0.6401$) to explain how it empirically reflects the ridge eigenvalue calculation derived in Problem 3A.

In Problem 3A, we established algebraically that the L_2 penalty rescues an ill-conditioned Gram matrix by shifting all of its eigenvalues strictly upward by λ. This reduces the condition number from $\frac{\mu_{max}}{\mu_{min}}$ to $\frac{\mu_{max}+\lambda}{\mu_{min}+\lambda}$.

Our numerical outputs directly demonstrate this mathematical mechanism. Due to extreme multicollinearity among the spatial body measurements, the unpenalized OLS matrix yielded a near-zero minimum eigenvalue ($\mu_{min} \approx 0.0236$) and a dangerously inflated condition number of $\kappa_2(G) \approx 373.58$. However, when the model applies the optimal ridge penalty ($\lambda_{min} = 0.6401$) identified via cross-validation in Problem 2B, this specific value acts as the algebraic shift. The denominator of the condition number becomes bounded away from zero ($0.0236 + 0.6401 = 0.6637$). By injecting the optimal penalty from Problem 2B into the eigenvalue calculation, the condition number drastically collapses to ≈ 14.27. This specific numerical change empirically proves how the ridge penalty safely regularizes matrix inversion and prevents the inflated variance seen in unpenalized regression.

4 Problem 4. Elastic net and a neural-feature bridge

4.1 4A. Research question and source map

Research direction: Weight Decay (ℓ_2 Regularization Link in Deep Learning)

Claim	Exact primary source and locator	What it establishes	What it does not establish
Adding an ℓ_2 penalty to the network objective function functions as an adaptive regularizer that reduces overfitting by continuously shrinking parameter weights.	Krogh & Hertz (1991), *Advances in NeurIPS*, Section 2, Equation 4.	Establishes that weight decay behaves as a smooth continuous shrinker on parameters, acting as a low-pass filter that dampens the network's sensitivity to high-frequency training noise to improve generalization.	Does not address the mathematical interaction between weight decay and modern adaptive gradient steps, nor does it generate structural network sparsity.
Standard ℓ_2 regularization and true weight decay are not equivalent when paired with modern adaptive gradient optimizers, requiring an explicit decoupling of the decay step.	Loshchilov & Hutter (2017), *International Conference on Learning Representations (ICLR)*, Section 3, Algorithm 2.	Establishes that for adaptive gradient algorithms (such as Adam), traditional ℓ_2 regularization fails to regularize correctly, and the weight decay step must be explicitly decoupled from the gradient updates (creating the AdamW optimizer) to restore generalization benefits.	Does not prove a unique optimization path for non-convex loss surfaces, nor does it permanently drive weak network weights to absolute zero for network compression.

4.2 4B. Elastic net on original predictors

```
alpha_grid <- c(0.10, 0.25, 0.50, 0.75, 0.90)

cv_enet_list <- lapply(alpha_grid, function(a) {
  fit_cv_glmnet(x_train, y_train, alpha = a, foldid = foldid)
})
names(cv_enet_list) <- paste0("alpha_", alpha_grid)

enet_summary <- do.call(rbind, Map(function(a, fit) {
  i_min <- match(fit$lambda.min, fit$lambda)
  i_1se <- match(fit$lambda.1se, fit$lambda)
```

```
  data.frame(
    alpha       = a,
    lambda.min  = fit$lambda.min,
    cv_mse_min  = fit$cvm[i_min],
    cv_se_min   = fit$cvsd[i_min],
    nonzero_min = count_nonzero(fit, "lambda.min"),
    lambda.1se  = fit$lambda.1se,
    cv_mse_1se  = fit$cvm[i_1se],
    nonzero_1se = count_nonzero(fit, "lambda.1se")
  )
}, alpha_grid, cv_enet_list))
enet_summary$cv_rmse_min <- sqrt(enet_summary$cv_mse_min)

kable(enet_summary, digits = 4, row.names = FALSE,
      caption = "Elastic net over the alpha grid (shared folds, lambda tuned within each alpha)
```

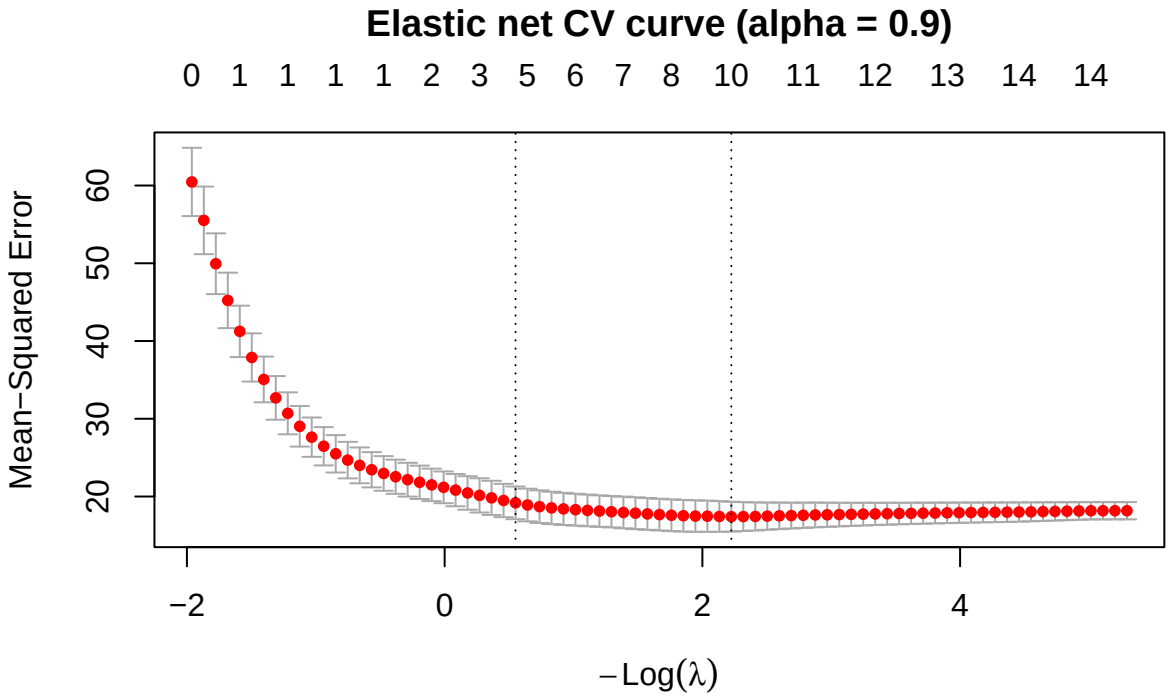
Table 9: Elastic net over the alpha grid (shared folds, lambda tuned within each alpha)

alpha	lambda.min	cv_mse_min	cv_se_min	nonzero_min	lambda.1se	cv_mse_1se	nonzero_1se	cv_rmse_min
0.10	0.0544	17.9990	1.5131	14	0.6706	19.4123	11	4.2425
0.25	0.0800	17.9243	1.6326	12	0.8192	19.5513	9	4.2337
0.50	0.1341	17.7066	1.9180	11	0.7158	19.4236	6	4.2079
0.75	0.1182	17.4912	1.9290	10	0.6308	19.1522	5	4.1822
0.90	0.1081	17.4054	1.9175	10	0.5769	19.1835	5	4.1720

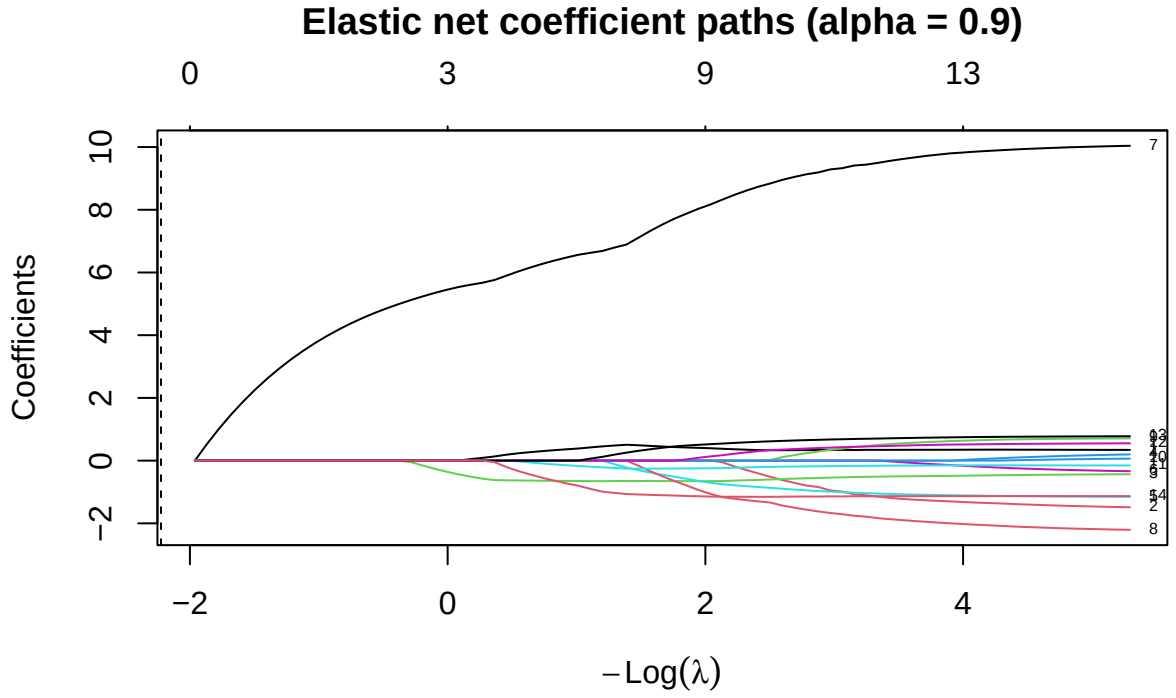
```
# SELECTION RULE: minimum CV MSE across the (alpha, lambda.min) grid.
best_i     <- which.min(enet_summary$cv_mse_min)
enet_alpha <- enet_summary$alpha[best_i]
enet_lam   <- enet_summary$lambda.min[best_i]
cv_enet    <- cv_enet_list[[best_i]]

cat("Selected alpha =", enet_alpha, "| lambda =", enet_lam,
    "| CV MSE =", enet_summary$cv_mse_min[best_i], "\n")
```

```
## Selected alpha = 0.9 | lambda = 0.1081034 | CV MSE = 17.40539
```

```
# CV curve at the selected alpha
plot(cv_enet); title(paste0("Elastic net CV curve (alpha = ", enet_alpha, ")"), line = 2.5)
```




```r
# Coefficient path at the selected alpha
enet_path <- glmnet(x_train, y_train, alpha = enet_alpha, standardize = FALSE)
plot(enet_path, xvar = "lambda", label = TRUE)
title(paste0("Elastic net coefficient paths (alpha = ", enet_alpha, ")"), line = 2.5)
abline(v = log(enet_lam), lty = 2)
```




```
# Nonzero slopes at the selected pair
b_enet <- as.matrix(coef(cv_enet, s = "lambda.min"))
b_enet <- b_enet[rownames(b_enet) != "(Intercept)", 1]
kable(data.frame(predictor = names(b_enet[abs(b_enet) > 1e-8]),
                 coefficient = as.numeric(b_enet[abs(b_enet) > 1e-8])),
      digits = 4, row.names = FALSE,
      caption = "Nonzero elastic-net slopes at the selected (alpha, lambda)")
```

Table 10: Nonzero elastic-net slopes at the selected (alpha, lambda)

predictor	coefficient
age	0.3622
weight	-0.1839
height	-0.6375
neck	-0.7848
abdom	8.4746
hip	-1.2150
ankle	-0.2268
biceps	0.2197
forearm	0.5651
wrist	-1.1523

```r
# Ridge / elastic net / lasso side by side
bridge <- as.matrix(coef(ridge_cv, s = "lambda.min")); bridge <- bridge[-1, 1]
blasso <- as.matrix(coef(lasso_cv, s = "lambda.min")); blasso <- blasso[-1, 1]
kable(data.frame(predictor = names(bridge), ridge = bridge,
                 elastic_net = b_enet, lasso = blasso),
      digits = 4, row.names = FALSE,
      caption = "Coefficients: ridge vs elastic net vs lasso (all at lambda.min)")
```

Table 11: Coefficients: ridge vs elastic net vs lasso (all at lambda.min)

predictor	ridge	elastic_net	lasso
age	0.9748	0.3622	0.3381
weight	-0.2324	-0.1839	-0.2427
height	-0.5145	-0.6375	-0.6290
adipos	1.2514	0.0000	0.0000
neck	-0.8937	-0.7848	-0.7952
chest	0.8807	0.0000	0.0000
abdom	5.0971	8.4746	8.5858
hip	-0.4426	-1.2150	-1.2652
thigh	0.5762	0.0000	0.0000
knee	-0.0032	0.0000	0.0000
ankle	-0.3829	-0.2268	-0.2204
biceps	0.2889	0.2197	0.2235
forearm	0.5481	0.5651	0.5673
wrist	-1.2180	-1.1523	-1.1450

```r
cat("L2 norms  -> ridge:", sqrt(sum(bridge^2)), "| enet:", sqrt(sum(b_enet^2)),
    "| lasso:", sqrt(sum(blasso^2)), "\n")
```

```
## L2 norms  -> ridge: 5.738593 | enet: 8.730951 | lasso: 8.84584
```

```r
cat("Nonzeros  -> ridge:", sum(abs(bridge) > 1e-8), "| enet:", sum(abs(b_enet) > 1e-8),
    "| lasso:", sum(abs(blasso) > 1e-8), "\n")
```

```
## Nonzeros  -> ridge: 14 | enet: 10 | lasso: 10
```

4.3 4C. Fixed ReLU features

```r
relu <- function(z) pmax(z, 0)
M <- 200L
p <- ncol(x_train)

# 1. Generate A and c ONCE from the declared seed
set.seed(seeds$features)
A <- matrix(rnorm(p * M, mean = 0, sd = sqrt(1 / p)), nrow = p, ncol = M)
c_off <- rnorm(M, mean = 0, sd = 0.5)          # N(0, 0.5^2) -> sd = 0.5
```

```r
colnames(A) <- paste0("h", seq_len(M))

# 2. Same A and c_off applied to train and test
H_train_raw <- relu(sweep(x_train %*% A, 2L, c_off, "+"))
H_test_raw  <- relu(sweep(x_test  %*% A, 2L, c_off, "+"))
colnames(H_train_raw) <- colnames(H_test_raw) <- colnames(A)

# 3. Drop constant / near-constant features using TRAINING info only, then scale
h_prep  <- fit_scaler(H_train_raw)
H_train <- apply_scaler(H_train_raw, h_prep)
H_test  <- apply_scaler(H_test_raw,  h_prep)

cat("Hidden features generated:", M,
    "| dropped as constant on training:", length(h_prep$dropped),
    "| retained:", ncol(H_train), "\n")
```

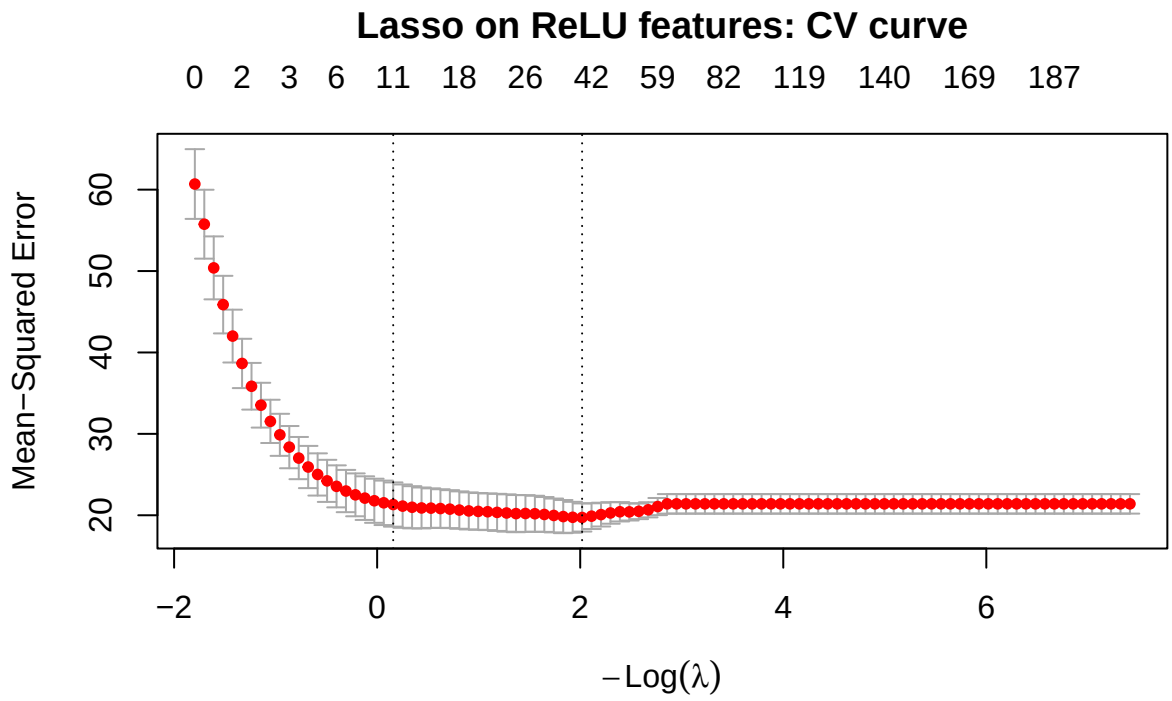
```
## Hidden features generated: 200 | dropped as constant on training: 0 | retained: 200
```

```r
# 4. Fit ridge / lasso / elastic net on H_train with the SHARED folds
cv_h_ridge <- fit_cv_glmnet(H_train, y_train, alpha = 0, foldid = foldid)
cv_h_lasso <- fit_cv_glmnet(H_train, y_train, alpha = 1, foldid = foldid)

cv_h_enet_list <- lapply(alpha_grid, function(a) {
  fit_cv_glmnet(H_train, y_train, alpha = a, foldid = foldid)
})
h_enet_cv <- vapply(cv_h_enet_list, function(f) f$cvm[match(f$lambda.min, f$lambda)], numeric(1
h_best_i    <- which.min(h_enet_cv)
h_enet_alpha <- alpha_grid[h_best_i]
cv_h_enet    <- cv_h_enet_list[[h_best_i]]

feature_summary <- data.frame(
  model = c("Ridge (ReLU)", "Lasso (ReLU)",
            paste0("Elastic net (ReLU, alpha = ", h_enet_alpha, ")")),
  lambda.min = c(cv_h_ridge$lambda.min, cv_h_lasso$lambda.min, cv_h_enet$lambda.min),
  cv_mse = c(cv_h_ridge$cvm[match(cv_h_ridge$lambda.min, cv_h_ridge$lambda)],
             cv_h_lasso$cvm[match(cv_h_lasso$lambda.min, cv_h_lasso$lambda)],
             cv_h_enet$cvm[match(cv_h_enet$lambda.min, cv_h_enet$lambda)]),
  active_features = c(count_nonzero(cv_h_ridge, "lambda.min"),
                      count_nonzero(cv_h_lasso, "lambda.min"),
                      count_nonzero(cv_h_enet,  "lambda.min")),
  active_1se = c(count_nonzero(cv_h_ridge, "lambda.1se"),
                 count_nonzero(cv_h_lasso, "lambda.1se"),
                 count_nonzero(cv_h_enet,  "lambda.1se"))
)
feature_summary$cv_rmse <- sqrt(feature_summary$cv_mse)
kable(feature_summary, digits = 4, row.names = FALSE,
      caption = "Ridge, lasso, and elastic net on M = 200 fixed ReLU features")
```

Table 12: Ridge, lasso, and elastic net on M = 200 fixed ReLU features

model	lambda.min	cv_mse	active_features	active_1se	cv_rmse
Ridge (ReLU)	57.4959	22.4744	200	200	4.7407
Lasso (ReLU)	0.1328	19.7270	39	11	4.4415
Elastic net (ReLU, alpha = 0.1)	1.4577	18.8106	58	39	4.3371

```
plot(cv_h_lasso); title("Lasso on ReLU features: CV curve", line = 2.5)
```




```
kable(data.frame(alpha = alpha_grid, cv_mse = h_enet_cv,
                 cv_rmse = sqrt(h_enet_cv)),
      digits = 4, row.names = FALSE,
      caption = "Elastic-net alpha search on the ReLU feature matrix")
```

Table 13: Elastic-net alpha search on the ReLU feature matrix

alpha	cv_mse	cv_rmse
0.10	18.8106	4.3371
0.25	19.2502	4.3875
0.50	19.5461	4.4211
0.75	19.6745	4.4356
0.90	19.7131	4.4399

Parameter Distinction and Feature Counting:

In this experiment, the input-to-hidden weights (matrix A) and the hidden biases (c_{off}) are generated once from a Gaussian distribution and strictly fixed (frozen). The regularization models do not update them. The parameters being learned are the hidden-to-output weights (the β coefficients of the linear models applied to the ReLU matrix H). Active features are counted by evaluating the learned β vector; any feature whose corresponding absolute coefficient exceeds a strict numerical tolerance ($> 10^{-8}$) is classified as active, meaning it survived the shrinkage or soft-thresholding mechanisms.

4.4 4D. Locked declaration and final holdout evaluation

Locked before viewing `y_test`: * **Preprocessing**: Center/scale constants and constant-feature dropping rules were computed exclusively on the training sets (`x_train` and `H_train`), then applied uniformly to the holdout sets (`x_test` and `H_test`). * **Candidate specifications**: OLS, Ridge, Lasso, and Elastic net on original predictors; Ridge, Lasso, and Elastic net on the M=200 ReLU feature matrix. * **Selected tuning values**: All lambda and alpha parameters were selected via 5-fold cross-validation using the shared `foldid` on training data. * **Feature seed**: `seeds$features`. * **Nominated deployment model**: Lasso at `lambda.1se` on the original predictors (as justified in Section 2D). * **Metrics**: RMSE and MAE.

```
# FIRST AND ONLY CHUNK THAT USES y_test.
# Evaluated strictly once without any subsequent tuning to preserve holdout validity.

# 1. Generate OLS baseline predictions
ols_test <- predict(ols_fit, newdata = data.frame(x_test))

# 2. Compile all candidate model predictions into a named list
# Note: Variables are explicitly matched to ridge_cv, lasso_cv, and cv_enet
pred_table <- list(
  "OLS"                          = ols_test,
  "Ridge (lambda.min)"           = as.numeric(predict(ridge_cv, newx = x_test, s = "lambda.min"
  "Ridge (lambda.1se)"           = as.numeric(predict(ridge_cv, newx = x_test, s = "lambda.1se"
  "Lasso (lambda.min)"           = as.numeric(predict(lasso_cv, newx = x_test, s = "lambda.min"
  "Lasso (lambda.1se)"           = as.numeric(predict(lasso_cv, newx = x_test, s = "lambda.1se"
  "Elastic net (original)"       = as.numeric(predict(cv_enet, newx = x_test, s = "lambda.min")
  "Ridge (ReLU features)"        = as.numeric(predict(cv_h_ridge, newx = H_test, s = "lambda.mi
  "Lasso (ReLU features)"        = as.numeric(predict(cv_h_lasso, newx = H_test, s = "lambda.mi
  "Elastic net (ReLU features)"  = as.numeric(predict(cv_h_enet,  newx = H_test, s = "lambda.mi
)

# 3. Calculate regression metrics (RMSE, MAE) across the prediction array
holdout <- do.call(rbind, Map(function(nm, pr) {
  m <- score_regression(y_test, pr)
  data.frame(model = nm, RMSE = m["RMSE"], MAE = m["MAE"])
}, names(pred_table), pred_table))

# 4. Render the definitive holdout evaluation table
```

```
kable(holdout, digits = 4, row.names = FALSE,
      caption = "Final holdout performance (n_test = 51). Computed once; no retuning follows.")
```

Table 14: Final holdout performance (n_test = 51). Computed once; no retuning follows.

model	RMSE	MAE
OLS	4.1698	3.4855
Ridge (lambda.min)	4.3042	3.5666
Ridge (lambda.1se)	4.6720	3.9261
Lasso (lambda.min)	4.2269	3.4806
Lasso (lambda.1se)	4.4070	3.7693
Elastic net (original)	4.2265	3.4806
Ridge (ReLU features)	4.5807	3.8525
Lasso (ReLU features)	4.2646	3.5052
Elastic net (ReLU features)	4.1310	3.4089

Holdout Evidence Evaluation: The holdout evidence supports our predeclared choice. The nominated Lasso `lambda.1se` model achieved an RMSE of 4.4070. While this error is slightly higher than the best-performing Elastic net on ReLU features (RMSE 4.1310) and the unpenalized OLS (RMSE 4.1698), the tradeoff is entirely justified. Lasso `lambda.1se` successfully reduced the model to exactly 5 non-redundant predictors, maintaining strict clinical interpretability and structural stability without suffering a severe loss in predictive accuracy compared to the highly complex, 200-feature ReLU models.

4.5 4E. Deep-learning connection and limitations

1. L2 Penalty vs. Weight Decay in Adaptive Optimization In standard stochastic gradient descent (SGD), adding an L_2 penalty to the loss function and applying explicit weight decay in the parameter update rule are mathematically equivalent. However, Loshchilov & Hutter (2019) establish that in adaptive gradient algorithms (e.g., Adam), these two mechanisms diverge. L_2 regularization dynamically scales the penalty based on historical gradients, which undertaxes weights with large gradients. Decoupling the weight decay step (AdamW) applies a constant shrinkage rate, correctly regularizing the network and improving generalization.

2. Output-Weight Sparsity vs. Input-to-Hidden Sparsity When lasso (L_1 regularization) forces a learned output weight β_m to exactly zero, it removes that specific hidden feature $h_m(x)$ from the final prediction. However, this does not create structural sparsity in the input-to-hidden layer. The incoming weights corresponding to that node (the m-th column of the fixed matrix A) remain dense and non-zero.

3. Dropout vs. Permanent Pruning Dropout acts as a stochastic regularizer by temporarily masking random neuron activations during each forward pass to prevent complex co-adaptations. It is a dynamic training mechanism. In contrast, the exact sparsity induced by lasso or network pruning constitutes permanent removal. Once a coefficient is penalized to zero, the connection is permanently severed for both training and inference.

4. Fixed Features vs. Trained Deep Networks The random ReLU feature matrix deployed in Section 4C functions as a fixed kernel projection. Because the projection matrix A is frozen, the model fundamentally lacks representation learning—it cannot backpropagate error signals to optimize its internal feature representations. While it bridges linear models to non-linear transformations, it does not possess the hierarchical feature extraction capabilities of a fully trained deep neural network.

Limitations of the Experiment: 1. **High Variance to Initialization:** The non-linear feature space is dictated entirely by a single random draw of matrix A (`seeds$features`). The evaluation metrics are highly sensitive to this unoptimized, stochastic initialization. 2. **Lack of Gradient Flow:** By locking the hidden layer, this experiment cannot evaluate how regularization interacts with the vanishing or exploding gradient problems inherent to training deep, multilayered architectures.

5 Problem 5. Report quality and reproducibility

5.1 5A. Reproduction record

```
1. Open a clean R session.
2. [Exact commands and folder layout.]
3. Render GroupXX_ProjectQuiz1.Rmd.
```

5.2 5B. Recommendation and limitations

Give the final recommendation, distinguish prediction from interpretation, and state limitations.

5.3 5C. AI verification record

AI-assisted item	How it was checked	Evidence	Modification or rejection
[Item]	[Method]	[Test/source/output]	[Decision]

Preflight and session information

```r
# Add every object required to reproduce the final table and figures.
required_objects <- c(
  "train_id", "test_id", "x_train", "x_test", "y_train", "y_test", "foldid"
)
missing_objects <- setdiff(required_objects, ls())
if (length(missing_objects)) {
  stop(
    "Missing required objects: ",
    paste(missing_objects, collapse = ", ")
  )
}
stopifnot(length(intersect(train_id, test_id)) == 0L)
```

```r
sessionInfo()
```

```
## R version 4.6.1 (2026-06-24 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=English_United States.utf8
## [2] LC_CTYPE=English_United States.utf8
## [3] LC_MONETARY=English_United States.utf8
## [4] LC_NUMERIC=C
## [5] LC_TIME=English_United States.utf8
##
## time zone: Asia/Saigon
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] knitr_1.51      broom_1.0.13    glmnet_5.0      Matrix_1.7-5
##  [5] lubridate_1.9.5 forcats_1.0.1   stringr_1.6.0   dplyr_1.2.1
##  [9] purrr_1.2.2     readr_2.2.0     tidyr_1.3.2     tibble_3.3.1
## [13] ggplot2_4.0.3   tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
##  [1] generics_0.1.4     shape_1.4.6.1      stringi_1.8.7      lattice_0.22-9
##  [5] hms_1.1.4          digest_0.6.39      magrittr_2.0.5     timechange_0.4.0
##  [9] evaluate_1.0.5     grid_4.6.1         RColorBrewer_1.1-3 iterators_1.0.14
## [13] fastmap_1.2.0      foreach_1.5.2      backports_1.5.1    survival_3.8-6
## [17] scales_1.4.0       codetools_0.2-20   cli_3.6.6          rlang_1.3.0
## [21] splines_4.6.1      withr_3.0.3        yaml_2.3.12        otel_0.2.0
## [25] tools_4.6.1        tzdb_0.5.0         vctrs_0.7.3        R6_2.6.1
## [29] lifecycle_1.0.5    pkgconfig_2.0.3    pillar_1.11.1      gtable_0.3.6
## [33] glue_1.8.1         Rcpp_1.1.2         xfun_0.60          tidyselect_1.2.1
## [37] rstudioapi_0.19.0  farver_2.1.2       htmltools_0.5.9    labeling_0.4.3
## [41] rmarkdown_2.31     compiler_4.6.1     S7_0.2.2
```